

Portable Vectorization and Parallelization of C++ Multi-dimensional Array Computations

Laurent Plagne and Kavoo Bojnourdi

PERICLES Dept,
EDF Lab Paris-Saclay,
7 Bld Gaspard Monge,
F-91120 Palaiseau, France
laurent.plagne@edf.fr, kavoo.bojnourdi@edf.fr

Abstract

This paper presents LEGOLAS++ arrays, a multi-dimensional array library. Parameterized type of LEGOLAS++ arrays enable data layout adaptation for specific Single Instruction Multiple Data (SIMD) core architectures. The mapping of complex array-based kernels to regular collections of data is efficiently vectorized. In addition, LEGOLAS++ arrays can combine multi-threaded parallelism with SIMD acceleration. For example, a direct tridiagonal solver applied to a collection of equally sized problems exhibits a speedup of more than $\times 22$ on an 8-core SIMD processor.

CCS Concepts • **Theory of computation** → **Vector / streaming algorithms**; • **Computing methodologies** → **Massively parallel and high-performance simulations**; *Parallel programming languages*; • **Software and its engineering** → **General programming languages**; • **General and reference** → *Performance*; • **Mathematics of computing** → *Mathematical software*

Keywords SIMD, multi-dimensional array, C++, vectorization, parallel programming, data layout transformation, template meta-programming.

1. Introduction

This paper presents a C++ multi-dimensional array class enabling portable vectorized and parallel implementation of complex computation kernels. This array class is part of our in-house LEGOLAS++ library dedicated to highly-structured sparse linear algebra [5].

The most innovative feature of LEGOLAS++ arrays is that their type enables data layout adaptation for specific SIMD core architectures. The mapping of complex array-based kernels to regular collections of data is efficiently vectorized. Independent applications of a given algorithm to a regular collection of data is a form of stream computing that happens frequently in scientific computations. OpenCL [7] offers a portable solution to express such kernel mapping that is valid on both GPUs and CPUs. One limitation of the OpenCL approach is that computation performance depends on the chosen data layout: the optimal data layout for GPU is not nec-

essarily the same as the optimal choice for CPU. This paper specifically addresses this data layout choice problem which is related to the SoA/AoS dilemma. We show that the data layout control of LEGOLAS++ arrays enables fine tuning for specific SIMD architectures while it maximizes the data locality of computational kernels. Due to the memory wall [9], this data locality is the dominant factor for performance.

Another aim of the paper is to illustrate the good interoperability of generic C++ libraries. LEGOLAS++ relies on the C++ Eigen library [4] that maps the computations on SIMD blocks and produces portable vectorized implementations. The explicit Eigen vectorization avoids reliance on C++ compiler automatic vectorization capabilities that may fail for complex computational kernels. In addition, the implementation of LEGOLAS++ arrays uses Intel C++ TBB [6] to combine multi-threaded parallelism to SIMD acceleration. As a result of this and the aid of modern C++ features like variadic templates and lambdas, the header-only implementation of LEGOLAS++ arrays is very compact and contains less than 2K lines of code.

The paper is organized as follows. In section 2 we shortly describe the LEGOLAS++ arrays. In section 3, a model problem consisting in multiple tridiagonal problems is described. Section 4 presents an optimal data layout for this problem and the use of the SIMD Eigen library. Section 5 describes the LEGOLAS++ arrays solution to the model problem and the corresponding performance. Conclusions and outlooks are given in section 6. The generality of the approach is discussed in this final section.

2. The LEGOLAS++ Array Class

The C++ LEGOLAS++ library addresses the problem of highly structured sparse linear algebra. Its main objective is to build and manipulate matrices which exhibit multiple block levels with different sparsity patterns at each level. Fig 1 represents the sparsity pattern of a 5-level blocked matrix that arises when solving a multi-group diffusion problem on a Cartesian mesh [5].

In this paper, we only consider the LEGOLAS++ subset dedicated to the manipulation of multi-dimensional arrays. Let

`Legolas :: Array<T,D>`

be a generic C++ array template class allowing the definition of D-dimensional arrays of Ts. In the remainder of the text, we will omit the `Legolas` namespace.

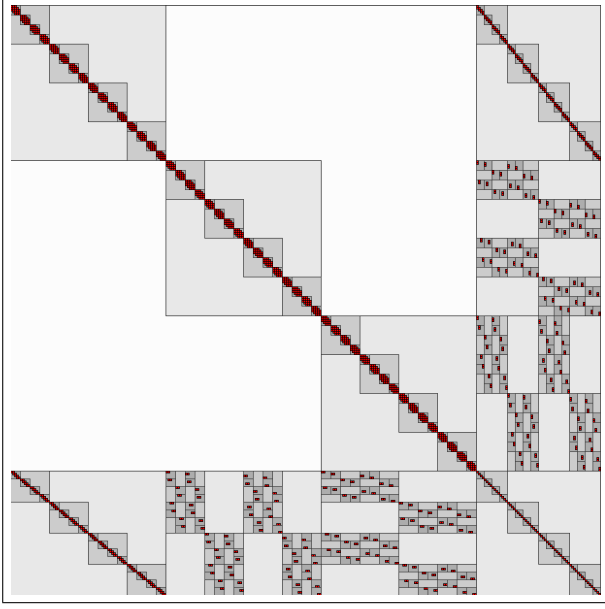


Figure 1. LEGOLAS++ 5-level Block Matrix Sparsity Pattern

Array<T,D> Nested Types	
Scalar	Array<T,D>::Scalar $\hat{=}$ T
Element	Array<T,D>::Element $\hat{=}$ Array<T,D-1> if $D > 1$ $\hat{=}$ T& if $D = 1$
Shape	int tuple for array sizes
Array<T,D> Methods	
Array(int n1,...,nD)	Ctor (variadic)
Element operator[]	Element accessor
int size()	returns the number of elements
Shape shape()	returns (n1,...,nD)

Table 1. Basic API of LEGOLAS++ Array<T,D> class

Table 1 summarizes the LEGOLAS++ Array API. The Array class implementation makes an extensive use of **recursive** template definitions. For example, the Array<T,D>::operator[] accessor returns:

- an array of inferior dimension (Array<T,D-1>) for $D > 1$,
- a scalar reference if $D == 1$.

This kind of static (compile time) branch is implemented via C++ template specializations.

The LEGOLAS++ Array class only handles *rectangular* arrays where all array elements are of equal size. The following code snippet illustrates the basic usage of LEGOLAS++ Array class.

```
//1D Array containing 10 double elts
Array<double,1> X1D(10);
//3D Array containing 100 double elts
Array<double,3> Y3D(10,5,2);
Array<double,2> Y2D=Y3D[0];
for (int k=0; k<10; k++)
  for (int j=0; j<5; j++)
    for (int i=0; i<2; i++)
      Y3D[k][j][i]=2.*Y3D[k][j][i]+1;
```

Figure 2. Cartesian mesh with coupling between neighboring Degrees of Freedom (DoFs) in the x direction. The DoFs (black arrows) are stored in the order defined by the circled numbers.

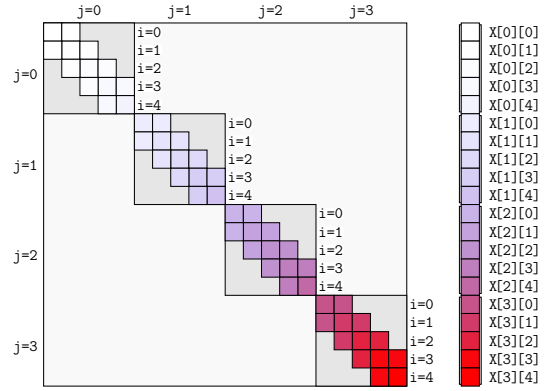
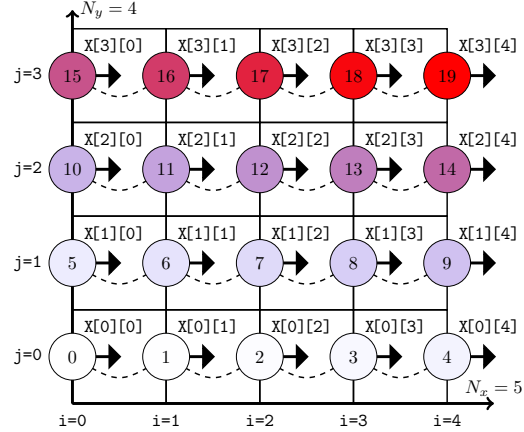


Figure 3. Description of matrix A: two-level block matrix structure Diagonal<Tridiagonal>.

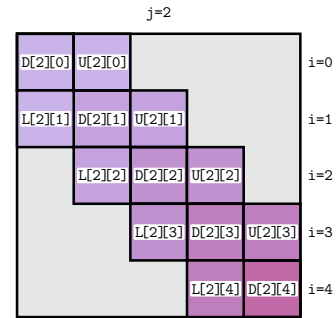


Figure 4. Description of matrix block A(2,2). A is described with 2D arrays U, L and D

3. The Running Example: A Collection of Independent Tridiagonal Systems

In this section we introduce a model problem consisting of multiple independent tridiagonal systems. This model problem will be used to illustrate the different aspects that should be considered for portable performances of a vectorized implementation.

Fig 2 represents a 2D Cartesian mesh, on which a given Partial Differential Equation (PDE) needs to be solved. Let us assume in this example that the PDE discretization uses Degrees of Freedom (DoFs) at the cell interfaces in the x direction, represented as black arrows in the figure. Many types of discretization techniques (such as Finite Differences or Finite Elements on structured meshes) yield coupling equations resembling:

$$-X_{i-1,j} + 2X_{i,j} - X_{i+1,j} = hB_{i,j}, \quad (1)$$

where we denote as X the DoFs array, B the right-hand side, and h the mesh size. We can see that only successive DoFs in each line are coupled (*i.e.* DoFs sharing the same j index). Fig 2 therefore represents 4 independent lines, each of them containing 5 coupled DoFs. Let $AX = B$ the corresponding linear system where A is the coupling matrix corresponding to Eq (1). Fig 3 represents this A matrix and X vector. We choose to represent X and B as 2D arrays. Let j be the coupling line index ($j \in [0, N_y = 4]$) and i the index inside each line ($i \in [0, N_x = 5]$). At first we choose to store contiguously the DoFs $X[j][i]$ belonging to lines j . This corresponds to the ordering defined by the circled numbers in Fig 2. This 2D array representation of the X vector leads to a 2-level block matrix A with diagonals $A(j,j)$ ($j \in [0, N_y = 4]$) containing tridiagonal blocks. Given that, in our discretization, the different diagonal blocks $A(j,j)$ are not coupled between them. Each diagonal block, *i.e.* tridiagonal system, can be solved independently. Each tridiagonal system can be solved via the Thomas algorithm [2]. A sequential C++ implementation is given on Listing 1. As illustrated in Fig 4, matrix A can be described by three 2D arrays U , L and D such that:

$$\begin{aligned} U[j][i] &= A(j,j)(i, i+1) \\ D[j][i] &= A(j,j)(i, i) \\ L[j][i] &= A(j,j)(i, i-1). \end{aligned}$$

In the example of Eq (1), we would have $D[j][i] = 2$ and $U[j][i] = L[j][i] = -1$.

```

1  for (int j=0 ; j<Ny ; j++){
2      typedef Array<double,1> A1D;
3      const A1D & D(D2D[j]),
4      U(U2D[j]), L(L2D[j]),
5      B(B2D[j]);
6      A1D & X(X2D[j]), S(S-);
7      //Begin of Thomas algorithm
8      double s(D[0]);
9      double sm1=1.f/s;
10
11     X[0]=B[0]*sm1;
12     //Forward
13     for (int i=1 ; i < Nx ; i++){
14         S[i]=U[i-1]*sm1;
15         s=D[i]-L[i]*S[i];
16         X[i]=B[i]-L[i]*X[i-1];
17         sm1=1./s;
18         X[i]*=sm1;
19     }
20     //Backward
21     for (int i=(Nx-2) ; i >= 0 ; i--){
22         X[i]-=S[i+1]*X[i+1];
23     } //End of Thomas algorithm
24 }

```

Listing 1. Thomas algorithm for each independent coupling lines $j \in [0, N_y]$.

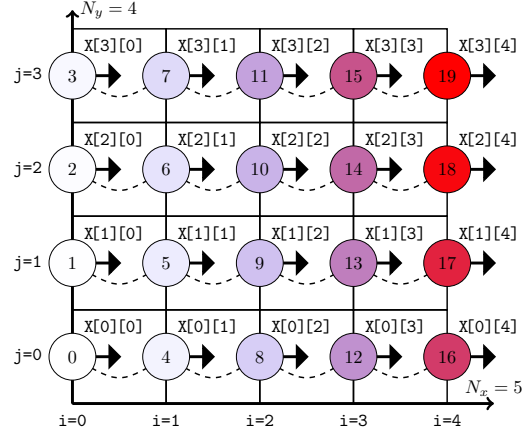


Figure 5. Transposition of the data layout. The DoFs (black arrows) are stored in the order defined by the circled numbers.

All coupling lines j are independent and the `for` loop at line 1 of the listing can be easily parallelized with OpenMP [3] or Intel TBB [6] for shared memory SMP and multi-core processors. Note that the Thomas algorithm corresponding to each coupling line (listing lines 8 to 23) is **not** parallel: the forward (lines 13 to 18) and backward (lines 21 to 23) steps must be processed sequentially and the corresponding `for` loops (lines 13 and 21) involve forward and backward dependencies between i indices (lines 14,15,16 and 22).

3.1 Vectorization via Data Layout Transposition

An efficient vectorization of Listing 1 is not straightforward. The sequential nature of the Thomas algorithm implies that vectorization can only be applied for processing multiple coupling lines j in parallel. SIMD capabilities of modern computing cores allow them to perform, at each clock cycle, several identical floating point operations (+,*,...) on different floating point values. These multiple operands must be stored in vector registers and these load/store operations can be very slow when one addresses non contiguous data in RAM. A solution can be to transpose the initial data layout to store contiguously the DoFs and matrix elements corresponding to successive j coupling lines. All the 2D arrays X , B , D U and L are transposed like:

$$X'[i][j] = X[j][i]$$

This transposition is represented in Fig 5 and the corresponding algorithm would exchange the outer (line 1) and the inner (lines 13 and 21) loops of Listing 1. Such a data layout transposition would transform our algorithm into a series of 1D-array operations. For example line 16 of Listing 1 is transformed into Listing 2.

3.2 Memory Wall, Arithmetic Intensity and Cache-Based Architectures

The loop corresponding to lines 8-9 of Listing 2 can be easily vectorized by C++ compilers. Despite the simplicity of this data layout transposition, this kind of implementation may exhibit disappointing performance for large enough problems on cache-based processor architectures.

Let us recall briefly the basics of the so-called *Roofline Model* [8] that gives a simple but effective way of predicting the performance of algorithms on present-time processor architectures. The arith-

```

1  for (int i=1 ; i < Nx ; i++) {
2      const A1D & L=L2D[i];
3      const A1D & B=L2D[i];
4      const A1D & Xprev=X2D[i-1];
5      A1D & X=X2D[i];
6      // Vectorizable Loop on j
7      // but low arithmetic intensity
8      for (int j=0 ; j<Ny ; j++){
9          X[j]=B[j]-L[j]*Xprev[j];
10     }

```

Listing 2. Transposed Data Layout: this code snippet corresponds to line 16 of Listing 1

metic intensity I_a for a given algorithm a is defined by:

$$I_a = \frac{\text{Number of floating points ops}}{\text{Number of RAM read and write ops}}.$$

On cache based architectures, this arithmetic intensity of a given algorithm plays a dominant role in its performance. Let C_p be the critical intensity characterizing a given processor architecture p . If $I_a > C_p$ then the algorithm is said to be *compute bound* and its performance is proportional to the computer Peak Performance. In the opposite case, $I_a < C_p$ and the algorithm is said to be *memory bound* and its performance is proportional to I_a . Since I_a does not count the *reads* and *writes* from the cache memory, its value depends on the actual data storage layout involved for a given algorithm. Let us consider a given square mesh size $n_x = n_y$ for which the data corresponding to a single coupling line j fills out the computer core cache. In this case, we can evaluate I_d the arithmetic intensity for the implementation described in Listing 1, where the 2D vectors are stored along x lines:

$$I_d = \frac{(5 \text{ *}) + (3 \text{ -}) + (1 \text{ /})}{5 \text{ read} + 1 \text{ write}} = \frac{13}{6},$$

where we assume the $/$ operation to be equivalent to 5 + operations. Let I_t be the arithmetic intensity corresponding to the transposed data layout (data stored along the y direction):

$$I_t = \frac{(5 \text{ *}) + (3 \text{ -}) + (1 \text{ /})}{10 \text{ read} + 2 \text{ write}} = \frac{13}{12},$$

Indeed, in the transposed case, the forward and the backward steps must be counted separately and the reads corresponding to an index i must be added to the ones corresponding to indices $i+1$ and $i-1$. Although the transposed data layout may enable an automatic vectorization, its halved arithmetic intensity eventually leads to a performance drop.

4. Interleaved Data Layout

On one hand, the original data layout (Fig 2) maximizes spatial and temporal data locality but prevents efficient vectorization. On the other hand, the transposed data layout (Fig 5) allows for automatic vectorization but reduces the data locality and consequently, the algorithm performance. In this section, we introduce an intermediate data layout which is compatible with vectorization while maintaining a good data locality.

Let us consider a processor with a SIMD unit that can compute p simultaneous floating point operations on contiguous data. For example, a processor core with SSE units (resp. AVX) can perform $p = 2$ (resp. $p = 4$) simultaneous operations on double precision floating point operands. The *interleaved* data layout consists in grouping successive j lines in packs of size p . This p packing can be achieved with LEGOLAS++ arrays. Let us transform the original

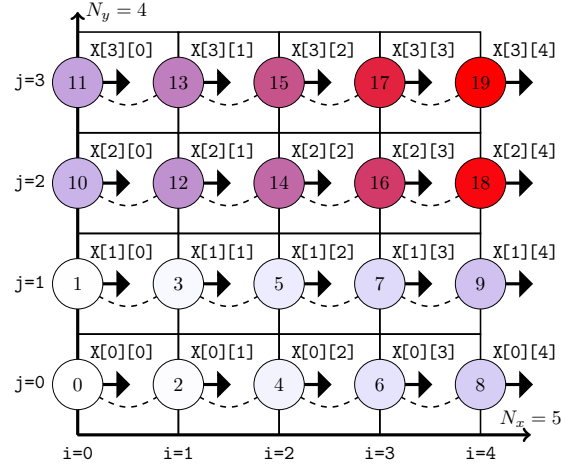


Figure 6. The coupling lines are grouped two by two. The DoF storage is *interleaved*: the DoFs (black arrows) are stored in the order defined by the circled numbers.

```

for (int jp=0 ; jp<Ny/p ; jp++){
    ...
    for (int i=1 ; i < Nx ; i++) {
        auto Xi=X[i]; auto Bi=B[i];
        auto Li=L[i]; auto Xim=X[i-1];
        for (int jc=0 ; jc < p ; jc++) {
            Xi[jc]=Bi[jc]-Li[jc]*Xim[jc];
        }
    }
}

```

Listing 3. Interleaved Data Layout: the Thomas algorithm now includes a third inner loop over the j_c components of 3D arrays.

2D arrays, $X[j][i]$, into a 3D array $Xp[jp][i][jc]$

$$\forall i \in [0, n_x[, \quad \forall j_p \in [0, n_y/p[, \quad \forall j_c \in [0, p[, \\ Xp[j_p][i][j_c] = X[j][i], \text{ with } j = p j_p + j_c,$$

where j_p stands for the *pack* index of j line indices while j_c stands for the *component* index inside this *pack*. Here we assume for simplicity that n_y is a multiple of p . Fig 6 represents such data layout transformation for the $p = 2$ case where j lines are grouped 2 by 2.

At this point one can see that the original algorithm of Listing 1 must be transformed to include a third inner loop with p iterations over the j_c components of 3D arrays. This transformation is illustrated in Listing 3. If p is known at compile time and corresponds to the SIMD width of the computing core, then inner loops over j_c can be fully vectorized by C++ compilers. Such a vectorizable and local implementation based on interleaved data layout should exhibit good performance but this approach has two drawbacks:

- our original algorithm is heavily transformed,
- we rely on C++ compiler vectorization capabilities.

```

for (int jp=0 ; jp<Ny/p ; jp++){
    auto U(Up[jp]), Lp(L2D[jp]), Dp(D2D[jp]);
    auto X(Xp[jp]), Bp(B2D[jp]), Sp(Sp);

    Block s(D[0]), sm1=1.f/s;
    // The remainder of the code is
    // identical to Listing 1 (line 11–24)
    // but the (+, -, /, *, =) operations
    // now take SIMD Block operands
    X[0]=B[0]*sm1;
    ...

```

Listing 4. Vectorized Thomas algorithm for each independent coupling pack of lines $j \in [0, N_y/p]$. Xp, Up, Lp, Dp, Bp are instances of `Array<Block, 2>`.

4.1 Explicit vectorization with the C++ Eigen Library

The C++ Eigen library [4] provides a very convenient way of expressing such SIMD operations with packed operands.

The following Eigen type contains p double precision floating point components:

```
typedef Eigen::Array<double, p, 1> Block;
```

Eigen overloads all the usual arithmetic operators, functions and expressions. For example:

```
Block a, b, c, d;    a+=b-c; a=b*c/d;
```

are transformed, at compile time, into the corresponding explicit intrinsic SIMD functions without any performance penalty compared to hand-coded assembly or intrinsic formulation. The floating point components of `Block` instances can be accessed via the following parenthesis operator:

```
Block a;    a(0)=1.0; a(1)=2.0;
```

Indeed 3D floating point arrays such as Xp can be interpreted as 2D arrays of Eigen Blocks. One can transform the type of Xp from 3D `double` array to 2D `Block` array;

```
// Array<double, 3> Xp(ny/p, nx, p);
Array<Block, 2> Xp(ny/p, nx);
```

By choosing a p parameter that corresponds to the particular SIMD width of a given architecture, the previous expression allows one to write portable SIMD expressions as follows:

```
Array<Block, 2> Xp(ny/p, nx),
                Yp(ny/p, nx);
Block a;
...
Xp[jp][i]+=a*Yp[jp][i];
```

which is compiled into SIMD instructions and is semantically equivalent to:

```
for (int jc=0 ; jc<p ; jc++)
    Xp[jp][i](jc)=a(jc)*Yp[jp][i](jc);
```

The Thomas algorithm based on `Array<Block, 2>` argument is described in Listing 4. The overloaded operations $+, -, *,$ defined for Eigen `Block` operands are automatically converted to vectorized SIMD instructions. Note that this implementation remains very close to the original code.

5. Portable Vectorization of Algorithms with LEGOLAS++ Arrays

We have seen in the previous section the role played by array data layout transformation in the vectorization process. The C++ Eigen library can be used to transform scalar algorithms into blocked algorithms where the standard operators (e.g. $+, -$ or $*$) are overloaded to deal with fixed sized blocks and generate appropriate SIMD instructions. In this section, we show that the parametric type of LEGOLAS++ arrays provide a convenient way to express both transformations.

The following template class:

```
Legolas::Array<T, D, P, DP>
```

accepts two extra integer parameters P and DP that control multi-dimensional array data layout. P specifies the packing factor p that must be applied to the dimension DP . For example:

```
// Array<double, 2> X(ny, nx);
Array<double, 2, p, 2> X(ny, nx);
Array<Block, 2> Xp=X.getPackedView();
```

Here X has the same constructor and shape as before but its data layout is packed on the outer dimension (y). The `getPackedView` method is used to recover Xp , the `Array<Block, 2>` view introduced in the previous section.

On top of the LEGOLAS++ parametric `Array` class, developers can write scalar algorithms that can be automatically vectorized via the LEGOLAS++ generic `map` function.

5.1 LEGOLAS++ Implementation of the Thomas algorithm

The initial listing 1 algorithm is rewritten as a C++ functor class that must define the `()` operator. The signature of this `()` operator must respect some constraints:

- the first two arguments must be two `int` defining the range of the parallel loop: `[begin, end]`;
- the rest of the arguments must be an arbitrary number of LEGOLAS++ arrays. Non array arguments can be passed via the functor constructor.

Listing 5 contains the LEGOLAS++ version of our running example.

5.2 LEGOLAS++ map Usage

Listing 6 contains the main program that will execute the `ranged` algorithm `ThomasSolver` for 800 coupling lines of length 200. On a dual socket Intel Xeon E6520 2.4 GHz totaling 2×4 cores (SSE4 support), the sequential scalar implementation performance reaches 0.6 GFlops. If one modifies type `A2D` to be `Array<float, 2, 4, 2>` in Listing 6, the SIMD performance now reaches 2.6 GFlops. Note that the `ThomasSolver` class is unchanged but the meaning of `Scalar` type has been modified by the application of the `map` function. Finally, Legolas provides the function `parmap` which combines multi-threaded parallelism with the vectorization. We now replace the `map` function by `parmap` in Listing 6. LEGOLAS++ delegates the parallelization machinery to the Intel TBB library. The performance reaches 13.3 GFlops. Table 2 summarizes these performance experiments.

5.3 LEGOLAS++ Array and map Implementation

In this section we present some details of the Legolas `Array` and `map` implementation. The C++ variadic templates introduced in C++11 and C++14 have shown to be a very effective tool for this kind of generic library implementation. Variadic templates allow for a varying number of function arguments. The `map` function

```

struct ThomasSolver{
template <class A2D>
void operator ()(int begin, int end,
                A2D D2D, A2D U2D, A2D L2D,
                A2D B2D, A2D X2D) const {
    typedef typename A2D::Element Element;
    typedef typename A2D::Scalar Scalar;
    Element S(X2D[0].shape());
    Scalar one(1.0), s, sml;

    for (int j=begin ; j<end ; j++){
        auto D=D2D[j];
        auto U=U2D[j]; auto L=L2D[j];
        auto B=B2D[j]; auto X=X2D[j];
        s=D[0];
        sml=one/s;
        const int size=X.size();
        //forward
        X[0]=B[0]*sml;
        for (int i=1; i<size; i++){
            S[i]=U[i-1]*sml;
            s=D[i]-L[i]*S[i];
            X[i]=B[i]-L[i]*X[i-1];
            sml=one/s;
            X[i]*=sml;
        }
        //backward
        for (int i=(size-2); i>=0 ; i--){
            X[i]-=S[i+1]*X[i+1];
        }
    }
};

```

Listing 5. LEGOLAS++ version of the Thomas algorithm for a collection of independent data.

```

int main () {

    size_t ny=800;
    size_t nx=200;

    typedef Array<float,2> A2D;
    //typedef Array<float,2,4,2> A2D;

    A2D u(ny, nx), l(ny, nx), d(ny, nx);
    A2D X(ny, nx), Y(ny, nx);
    //... Arrays initialization

    map(ThomasSolver(), d, u, l, Y, X);
    //parmap(ThomasSolver(), d, u, l, Y, X);

}

```

Listing 6. Legolas map of the Thomas algorithm (0.6 GFlops)

call	Array	Time(s)	GFlops	Speed-Up
map	<float,2>	0.88	0.60	1.0
parmap	<float,2>	0.13	4.2	×6.97
map	<float,2,4,2>	0.20	2.6	×4.31
parmap	<float,2,4,2>	0.04	13.3	×22.1

Table 2. Performance summary for $n_x = 200$, $n_y = 800$ (Intel Xeon E6520 2.4 GHz with 2×4 cores)

```

1 template <class ALGO, typename ... ARRAYS>
2 void map(ALGO a, ARRAYS... rest){
3
4     int s=getArraysSize(rest...);
5     int p=getPackSize(rest...);
6     int sp=getPackedArraysSize(rest...);
7
8     // Vectorized part:
9     // algo a in [O, sp=s/p[
10    a(0, sp, rest.getPackedView()...);
11
12    // Scalar remainder
13    // scalar algo a in [sp*p, s[
14    a(sp*p, s, rest...);
15
16 }

```

Listing 7. Vectorized map implementation

```

template <class ALGO, typename ... ARRAYS>
void parmap(int begin, int end,
            ALGO algo, ARRAYS... rest){

    tbb::parallel_for(
        tbb::blocked_range<intint

```

Listing 8. Parallel parmap implementation

takes an algorithm that can have an arbitrary number of array arguments.

The `getPackedView` method is used for the vectorized implementation of the `map` function as shown in Listing 7. Line 4, the variable `s` stores the size of the array (e.g. $s = 9$). The pack size `p` (e.g. $p = 4$) is retrieved line 5. Line 6, the size `sp` of the corresponding `PackedView` is evaluated (e.g. $sp = 9/4 = 2$). The algorithm is first called with the SIMD view of all array arguments line 10. Note how the `getPackedView` method is applied to the arbitrary number of array arguments `rest`. Finally, if `s` is not a multiple of `p`, then algorithm `a` is applied (line 14) to the remaining elements, ranging in $[sp * p, s[$.

A final example that illustrates the great expressivity of variadic templates is given in Listing 8 where C++ lambdas are mixed with variadic templates for the `parmap` implementation.

6. Conclusion and Outlooks

We have presented a small set of C++ classes that allows developers to create and manipulate N-dimensional arrays. The type of these arrays allows developers to get some control on the underlying array data storage. This data layout manipulation combined with the Eigen C++ library enable auto-vectorization of a large class of algorithms that perform similar computations over a range of arrays. The incorporation of the C++ Intel TBB Library combines the shared memory parallelism with SIMD computations. As a result, our example computation, consisting of multiple resolutions of tridiagonal systems, exhibits a speed-up of $\times 22$ on our 8-core computer. The C++14 variadic templates are shown to be a very effective tool for building a highly generic and recursive library.

6.1 Generality of the Approach

In order to discuss the generality of this approach, let us describe more precisely its application domain:

- Let $V = \{v_1, v_2, \dots, v_n\}$ be a dataset consisting of n variables where each v_i can be a scalar value or a multi-dimensional array of scalars (not necessarily of the same dimension).
- Let $a(V)$ be an algorithm to be applied to a dataset V . Such an algorithm will be said to be *deterministic* if it consists in a series of operations that do not depend on the data values v_i (no control branch).
- Let $W = \{V_1, V_2, \dots, V_m\}$ be a set of m datasets. W will be said to be *regular* if all datasets V_j are of the same size n and, denoting by $v_{i,j}$ the i^{th} variable of the j^{th} dataset V_j ,
 $\forall j, j' \in [1, m]^2, \forall i \in [1, n], \text{size}(v_{i,j}) = \text{size}(v_{i,j'})$.

The proposed method can be applied to vectorize (and parallelize) the application of any *deterministic* algorithm to any *regular* set of dataset. Let us describe our running example with this vocabulary:

- The Thomas algorithm applied to each coupling line is the *deterministic* algorithm a .
- The 1D arrays L, D, U, X, B and the scalar S form a dataset V_j for the coupling line j .
- $W = \{V_j\}$ is *regular* because all lines have the same size.

With this formal presentation of the application domain of the proposed approach, one can think of applications outside our original linear algebra domain. For example, in the image processing domain: one can consider a given filter, not parallel, to be applied to a collection of equally sized images. In this case, the scalar implementation of this filter may be automatically vectorized via the proposed LEGOLAS++ arrays and maps.

6.2 Outlooks

Future developments could extend array packing to several dimensions. This could become a convenient way to express tile-based computations. Although the presented implementation is in C++, we believe that the underlying concepts of data layout manipulation and SIMD blocked operations could be useful in other contexts. The Julia language [1], under active development, puts a strong emphasis on multi-dimensional arrays and could therefore be an appropriate context for this work. The very powerful meta-programming feature of Julia should be very efficient in this respect.

We would like to thank the reviewers for their evaluation and detailed comments that helped us to improve this manuscript.

References

- [1] J. Bezanson, S. Karpinski, V. Shah, and A. Edelman. Julia: A fast dynamic language for technical computing. In *Lang.NEXT*, Apr. 2012. URL <http://julialang.org/images/lang.next.pdf>.
- [2] S. D. Conte. C. deBoor. *Elementary Numerical Analysis: An Algorithmic Approach*, 3, 1972.
- [3] L. Dagum and R. Menon. OpenMP: an industry standard api for shared-memory programming. *Computational Science & Engineering, IEEE*, 5(1):46–55, 1998.
- [4] G. Guennebaud, B. Jacob, et al. Eigen v3, 2010. URL <http://eigen.tuxfamily.org>.
- [5] W. Kirschenmann, L. Plagne, A. Ponçot, and S. Vialle. Parallel SPN on multi-core CPUs and many-core GPUs. *Transport Theory and Statistical Physics*, 39(2-4):255–281, 2011.
- [6] A. Robison, M. Voss, and A. Kukanov. Optimization via reflection on work stealing in tbb. In *Parallel and Distributed Processing, 2008. IPDPS 2008. IEEE International Symposium on*, pages 1–8. IEEE, 2008.
- [7] J. E. Stone, D. Gohara, and G. Shi. OpenCL: A parallel programming standard for heterogeneous computing systems. *IEEE Des. Test*, 12(3):66–73, May 2010. ISSN 0740-7475. doi: 10.1109/MCSE.2010.69. URL <http://dx.doi.org/10.1109/MCSE.2010.69>.
- [8] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [9] W. A. Wulf and S. A. McKee. Hitting the memory wall: implications of the obvious. *ACM SIGARCH computer architecture news*, 23(1):20–24, 1995.